

Summer Internship Training

(June 2026-July 2026)

on

Mastering Data Structures using Java Concepts to Real-world Applications

Submitted by

Pericharla Hari Vignesh Registration Number: 12418833

Under the Guidance of

**Dr. OmPrakash Yadav Associate
Professor**

School of Computer Science and Engineering



**L OVELY
P ROFESSIONAL
U NIVERSITY**

Abstract

Parking management has become a significant challenge in residential complexes, shopping malls, office buildings, hospitals, educational institutions, and public parking facilities due to the increasing number of vehicles. Manual parking management methods often lead to inefficient slot allocation, longer waiting times, inaccurate record keeping, and difficulty in locating parked vehicles. These issues reduce operational efficiency and create inconvenience for both vehicle owners and parking administrators. A reliable, automated parking management system is therefore essential for organizing parking operations while minimizing human effort.

The **Smart Parking Management System** is a standalone Java console application designed to manage vehicle parking efficiently in an offline environment. The system allows users to park vehicles, allocate the nearest available parking slot, search parked vehicles instantly, remove vehicles upon exit, calculate parking fees based on parking duration, and maintain complete parking history. The application operates without requiring any external database, web server, or internet connection, making it suitable for academic learning as well as small-scale parking facilities.

The project demonstrates the practical application of Data Structures and Algorithms (DSA) in solving a real-world problem. A Queue manages available parking slots and waiting vehicles using the First-In-First-Out (FIFO) principle. A HashMap enables near constant-time retrieval of vehicle information using the vehicle registration number as the key. A LinkedList stores completed parking records to maintain parking history efficiently, while a Stack can be used to implement an Undo feature that restores the most recent parking or vehicle exit operation. Additionally, classical searching and sorting algorithms such as Linear Search, Binary Search, Bubble Sort, Insertion Sort, and Merge Sort are incorporated to organize and retrieve parking records efficiently.

The project is developed entirely using core Java and emphasizes implementing data structures from fundamental concepts rather than relying solely on built-in library implementations. Local file storage is used to preserve parking records between application sessions. The Smart Parking Management System demonstrates how classical DSA concepts can be integrated into a practical software solution that is efficient, maintainable, and suitable for real-world parking management scenarios.

Keywords

Smart Parking Management System, Java Console Application, Data Structures, Queue, HashMap, LinkedList, Stack, Searching Algorithms, Sorting Algorithms, Parking Slot Allocation, Vehicle Management, Parking Fee Calculation, Offline Application.

Contents

1. Introduction

1.1 Scope

1.2 Stakeholders

- 2. Problem Statement and Objectives**
 - 2.1 Problem Statement
 - 2.2 Objectives
 - 2.3 Feasibility
- 3. Requirement Analysis**
 - 3.1 Functional Requirements
 - 3.2 Non-functional Requirements
- 4. System Design**
 - 4.1 Architecture
 - 4.2 Data Model
 - 4.3 Workflow
- 5. Data Structures and Algorithms**
 - 5.1 Queue for Parking Slot Allocation and Waiting Vehicles
 - 5.2 HashMap for Vehicle Search
 - 5.3 LinkedList for Parking History
 - 5.4 Stack for Undo Operations
 - 5.5 Sorting Algorithms
 - 5.6 Searching Algorithms
 - 5.7 Complexity Summary
- 6. Implementation Details**
 - 6.1 Technology Used
 - 6.2 Module Overview
 - 6.3 Front-end Concept
- 7. Testing and Results**
 - 7.1 Test Cases
 - 7.2 Observations
- 8. Limitations and Future Scope**
 - 8.1 Current Limitations
 - 8.2 Future Enhancements
- 9. Conclusion**
- 10. References**

1. Introduction

Parking management has become increasingly important due to the growing number of vehicles in residential complexes, shopping malls, offices, hospitals, and educational institutions. Manual parking systems often lead to inefficient slot allocation, difficulty in locating parked vehicles, longer waiting times, and inaccurate record keeping. An efficient parking management system should therefore provide quick parking allocation, fast vehicle search, proper record management, and accurate parking fee calculation.

The Smart Parking Management System is a menu-driven Java console application developed to simplify parking operations. It allows users to park vehicles, allocate available parking slots, search vehicles using their registration numbers, remove vehicles during exit, calculate parking fees, and maintain parking history. The application uses Data Structures such as Queue, HashMap, LinkedList, and Stack along with searching and sorting algorithms to improve the efficiency of parking operations. The system works completely offline and requires only the Java Development Kit for execution.

1.1 Scope

- Park and remove vehicles efficiently.
- Allocate available parking slots automatically.
- Search parked vehicles using registration numbers.
- Display available and occupied parking slots.
- Calculate parking fees based on parking duration.
- Maintain parking history and generate simple reports.

1.2 Stakeholders

Stakeholder	Role	SystemValue
Vehicle Owner	Parks and retrieves vehicles	Quick parking and easy vehicle search
Parking Administrator	Manages daily parking operations	Efficient parking management
Parking Facility	Monitors parking usage	Better utilization of parking spaces
Instructor / Evaluator	Reviews the project	Evaluates practical implementation of DSA

2. ProblemStatementandObjectives

2.1ProblemStatement

Traditional parking management systems often rely on manual processes or basic record keeping, making it difficult to allocate parking slots efficiently, locate parked vehicles quickly, and maintain accurate parking records. During busy hours, finding an available slot and managing vehicle entries and exits becomes time-consuming. The problem is to develop a lightweight, offline system that simplifies parking management, provides fast vehicle search, and organizes parking operations using suitable data structures.

2.2 Objectives

1. Develop an offline Java console application for parking management.
2. Allocate parking slots efficiently to incoming vehicles.
3. Search parked vehicles quickly using their registration numbers.
4. Calculate parking fees based on parking duration.
5. Maintain parking history for future reference.
6. Implement Queue, HashMap, LinkedList, and Stack to perform different parking operations efficiently.
7. Store parking records locally so that data is available between sessions.

2.3 Feasibility

The project is technically feasible because it requires only the Java Development Kit and does not depend on external frameworks, databases, or internet connectivity. It is operationally feasible for academic demonstrations and small parking facilities, as users can easily operate the menu-driven console application. The offline approach keeps the implementation simple while focusing on the effective use of Data Structures and Algorithms.

3.RequirementAnalysis

3.1FunctionalRequirements

ID	Requirement	Description
FR-01	Park vehicle	Capture vehicle details and allocate an available parking slot.
FR-02	Search vehicle	Search for a parked vehicle using its registration number.
FR-03	Remove vehicle	Remove a vehicle during exit and free the parking slot.
ID	Requirement	Description
FR-04	Display parking status	Show available and occupied parking slots.
FR-05	Calculate parking fee	Calculate parking charges based on parking duration.
FR-06	Parking history	Maintain records of parked and exited vehicles.
FR-07	Waiting queue	Add vehicles to a waiting queue when parking is full.
FR-08	Local persistence	Save and reload parking records from a local file.

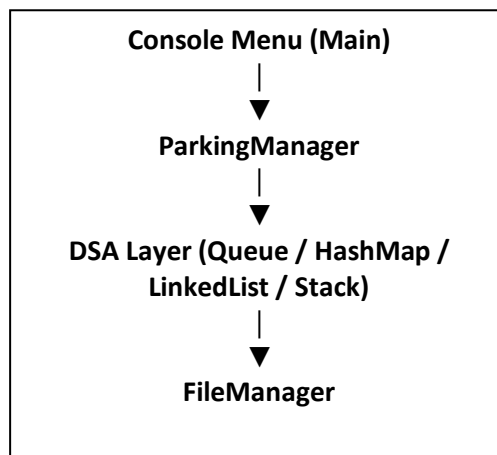
3.2Non-functionalRequirements

- Offline operation: the application works without internet or an external database.
- Usability: a simple menu-driven console interface allows users to perform all operations.
- Performance: vehicle search and parking operations are completed efficiently using suitable data structures.
- Reliability: parking records are stored locally to prevent data loss between sessions.
- Maintainability: each module, such as parking management, fee calculation, searching, and storage, is implemented separately for easier maintenance.

4.SystemDesign

4.1Architecture

The Smart Parking Management System follows a simple layered architecture. The presentation layer consists of a menu-driven console interface that accepts user input and displays results. The application layer, ParkingManager, handles parking operations such as slot allocation, vehicle search, fee calculation, and parking history. The DSA layer manages the Queue, HashMap, LinkedList, and Stack, while the FileManager handles saving and loading records from a local file. This design keeps the user interface independent from the application logic and data storage.



4.2 Application Data Model

Field	Type	Purpose
Vehicle Number	String	Unique vehicle registration number
Owner Name	String	Name of the vehicle owner
Vehicle Type	String	Type of vehicle (Bike, Car, Truck)
Slot Number	int	Allocated parking slot
Entry Time	LocalDateTime	Vehicle entry time
Exit Time	LocalDateTime	Vehicle exit time

4.3 Workflow

The user enters vehicle details and the system checks for an available parking slot. If a slot is available, it is allocated to the vehicle and the details are stored in memory and saved locally. During exit, the system searches for the vehicle, calculates the parking fee, frees the parking slot, updates the parking history, and saves the updated records. If the parking area is full, incoming vehicles are placed in a waiting queue until a slot becomes available.



5.DataStructuresandAlgorithms

5.1Queue for Parking Slot Allocation

A Queue is used to manage available parking slots and waiting vehicles. When a vehicle enters, the first available slot is allocated. If the parking area is full, incoming vehicles are added to the waiting queue and assigned a slot in the order they arrive

5.2HashMap for Vehicle Search

A HashMap stores vehicle details using the vehicle registration number as the key. This allows the system to search for parked vehicles quickly and retrieve their information efficiently.

5.3LinkedList for Parking History

A LinkedList is used to maintain the history of parked and exited vehicles. It allows records to be added efficiently as vehicles complete their parking sessions

5.4Stack for Undo Operation

A Stack is used to store the most recent parking operations. If a user performs an incorrect parking or exit operation, the last action can be undone using the Last-In-First-Out (LIFO) principle.

5.5ComplexitySummary

Operation	Data Structure / Algorithm	Time Complexity
Allocate parking slot	Queue	O(1)
Search vehicle	HashMap	O(1)
Maintain parking history	LinkedList	O(1)
Undo last operation	Stack	O(1)
Linear search	Linear Search	O(n)
Binary search	Binary Search	O(log n)
Bubble sort	Bubble Sort	O(n ²)

6.ImplementationDetails

6.1TechnologyUsed

The Smart Parking Management System is developed using Core Java (JDK) without any external frameworks or libraries. The application uses *java.util.Scanner* for user input, *java.time.LocalDateTime* for managing entry and exit times, and Java file handling classes for storing parking records locally. The project is implemented as a console-based application to emphasize the use of Data Structures and Algorithms.

6.2 Module Overview

- Vehicle – Stores information such as vehicle number, owner name, vehicle type, and parking details.
- Parking Manager – Handles parking operations, slot allocation, vehicle search, parking history, and fee calculation.
- Fee Calculator – Calculates parking charges based on parking duration.
- Search – Implements Linear Search and Binary Search for retrieving vehicle records.
- Sort – Implements Bubble Sort, Insertion Sort, and Merge Sort for organizing parking records.
- File Manager – Saves and loads parking records from a local file.
- Main – Provides the menu-driven console interface and coordinates all modules.

Figure 6.1 Dashboard:



```
<section class="page active" id="page-dashboard">
  <div class="page-head">
    <div>
      <h1>Operations Overview</h1>
      <p>Live status of every slot, vehicle and rupee moving through the lot.</p>
    </div>
    <div class="page-head-actions">
      <span class="live-pill"><span class="live-dot"></span> Live · Auto-refreshing</span>
      <button class="btn btn-primary"><i class="fa-solid fa-plus"></i> New Entry</button>
    </div>
  </div>
  <div class="kpi-grid">
```

Figure 6.2 Vehicle Entry:

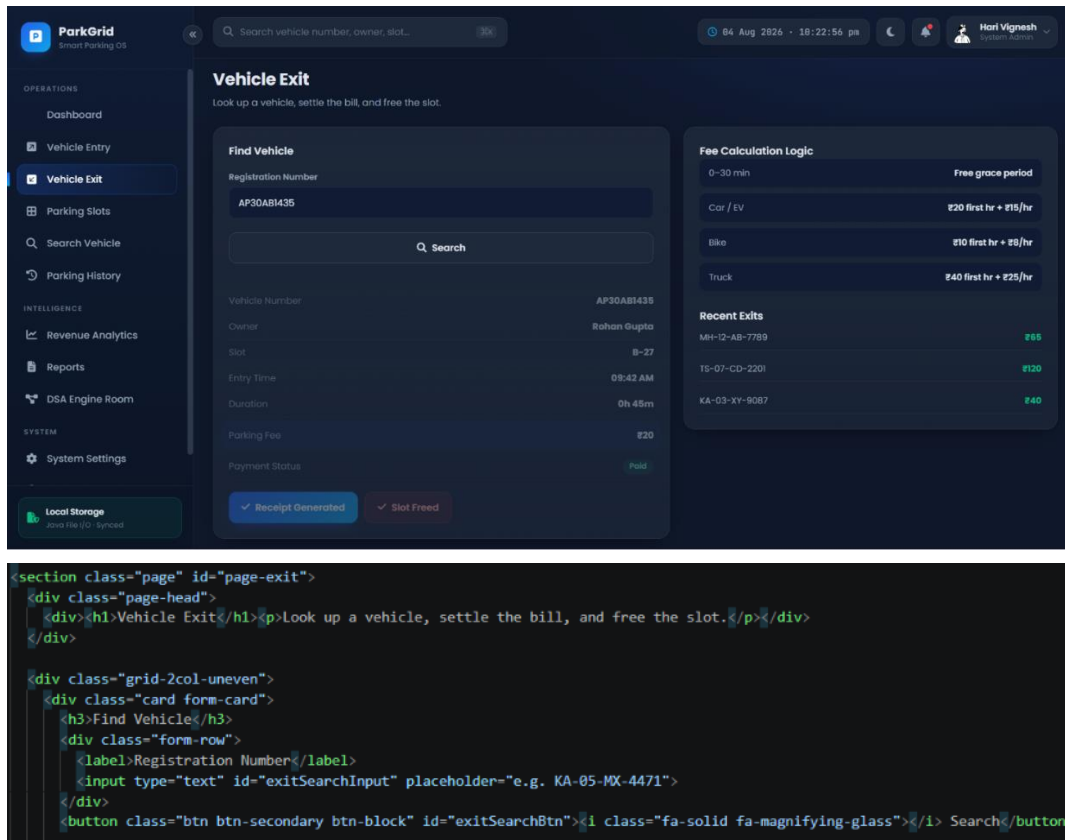
```

<section class="page" id="page-entry">
  <div class="page-head">
    <div><h1>Vehicle Entry</h1><p>Register an incoming vehicle and auto-allocate the nearest free slot.</p></div>
  </div>

  <div class="grid-2col-uneven">
    <div class="card form-card">
      <h3>Entry Details</h3>
      <form id="entryForm">
        <div class="form-row">
          <label>Vehicle Number</label>
          <input type="text" placeholder="e.g. TS-09-EA-1234" id="entryVehicleNo" required>
        </div>
        <div class="form-row">
          <label>Owner Name</label>
          <input type="text" placeholder="Full name" id="entryOwner" required>
        </div>
        <div class="form-row-split">
          <div class="form-row">
            <label>Vehicle Type</label>
            <select id="entryType">
              <option>Car</option><option>Bike</option><option>Truck</option><option>EV</option>
            </select>
          </div>
          <div class="form-row">
            <label>Phone Number</label>
            <input type="text" placeholder="10-digit mobile" id="entryPhone">
          </div>
        </div>
        <div class="allocate-box">
          <i class="fa-solid fa-wand-magic-sparkles"></i>
          <div>
            <b>Nearest slot auto-allocated by Queue DSA</b>
            <small>Front of the free-slot queue is dequeued instantly on submit</small>
          </div>
        </div>
        <div class="form-row">
          <label>Entry Time</label>
          <input type="text" id="entryTime" readonly>
        </div>
        <button type="submit" class="btn btn-primary btn-block"><i class="fa-solid fa-ticket"></i> Generate Parking Ticket</button>
      </form>
    </div>
  </div>

```

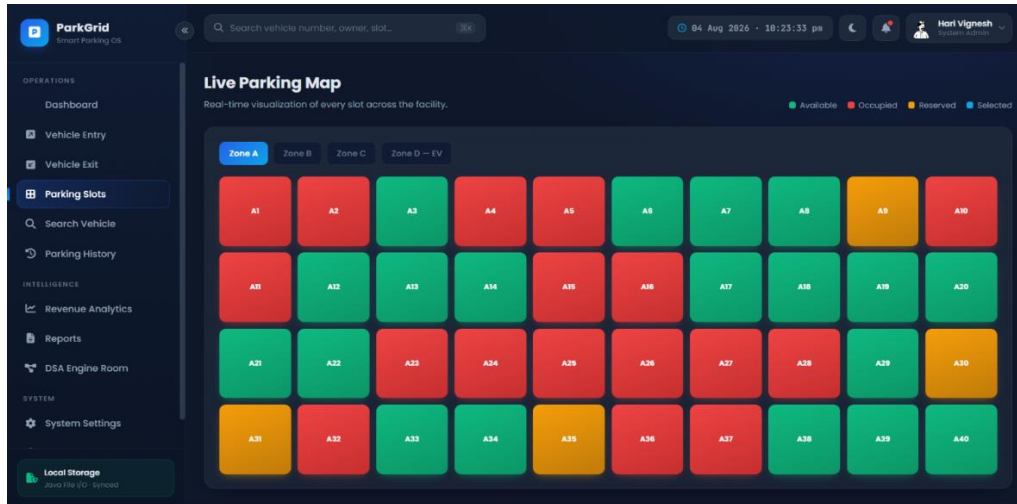
Figure 6.3 Vehicle Exit:



6.3 Front-end Concept

A simple console-based interface is provided for user interaction through a numbered menu. Users can perform operations such as parking a vehicle, removing a vehicle, searching vehicle details, viewing parking status, checking parking history, and generating reports. In the future, the application can be extended with a graphical user interface using Java Swing or JavaFX while retaining the existing backend logic.

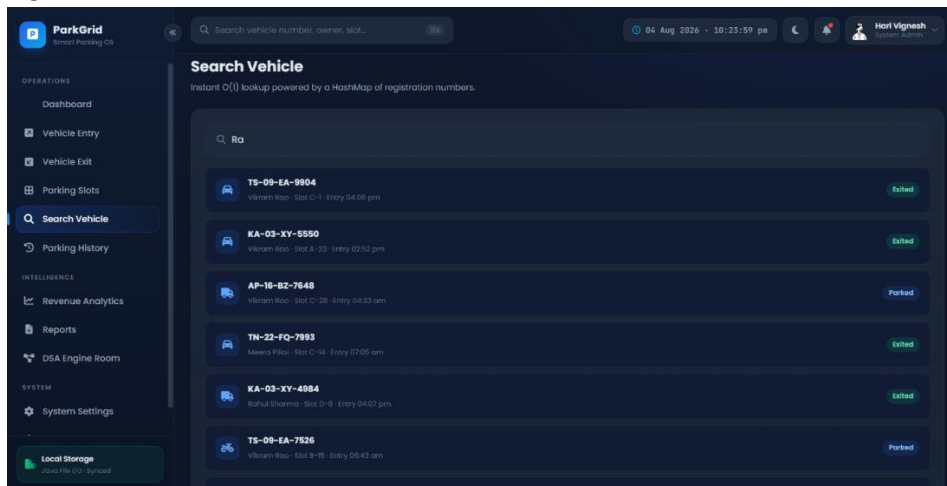
Figure 6.4 Parking Slots:



```
<section class="page" id="page-slots">
  <div class="page-head">
    <div><h1>Live Parking Map</h1><p>Real-time visualization of every slot across the facility.</p></div>
    <div class="slot-legend">
      <span><i style="background:var(--success)"></i> Available</span>
      <span><i style="background:var(--danger)"></i> Occupied</span>
      <span><i style="background:var(--warning)"></i> Reserved</span>
      <span><i style="background:var(--secondary)"></i> Selected</span>
    </div>
  </div>

  <div class="card">
    <div class="zone-tabs">
      <button class="active" data-zone="A">Zone A</button>
      <button data-zone="B">Zone B</button>
      <button data-zone="C">Zone C</button>
      <button data-zone="D">Zone D - EV</button>
    </div>
    <div class="slot-grid" id="slotGrid"></div>
  </div>
</section>
```

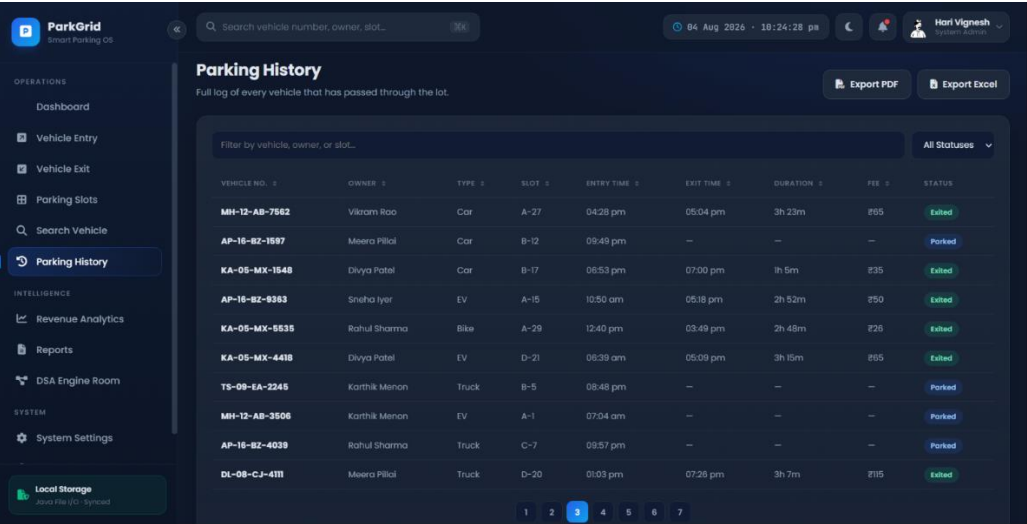
Figure 6.5 Search Vehicle:



```
<section class="page" id="page-search">
  <div class="page-head"><div><h1>Search Vehicle</h1><p>Instant O(1) lookup powered by a HashMap of registration numbers.</p></div></div>

  <div class="card">
    <div class="search-bar big">
      <i class="fa-solid fa-magnifying-glass"></i>
      <input type="text" id="vSearchInput" placeholder="Type a vehicle number_ e.g. TS-09">
    </div>
    <div class="search-results" id="vSearchResults"></div>
  </div>
</section>
```

Figure 6.6 Parking History:



```
<section class="page" id="page-history">
  <div class="page-head">
    <div><h1>Parking History</h1><p>Full log of every vehicle that has passed through the lot.</p></div>
    <div class="page-head-actions">
      <button class="btn btn-secondary"><i class="fa-solid fa-file-pdf"></i> Export PDF</button>
      <button class="btn btn-secondary"><i class="fa-solid fa-file-excel"></i> Export Excel</button>
    </div>
  </div>

  <div class="card">
    <div class="table-toolbar">
      <input type="text" id="historyFilter" placeholder="Filter by vehicle, owner, or slot_">
      <select id="historyStatusFilter">
        <option value="">All Statuses</option>
        <option value="Parked">Parked</option>
        <option value="Exited">Exited</option>
      </select>
    </div>
  </div>
```

Figure 6.7 Revenue Analytics:



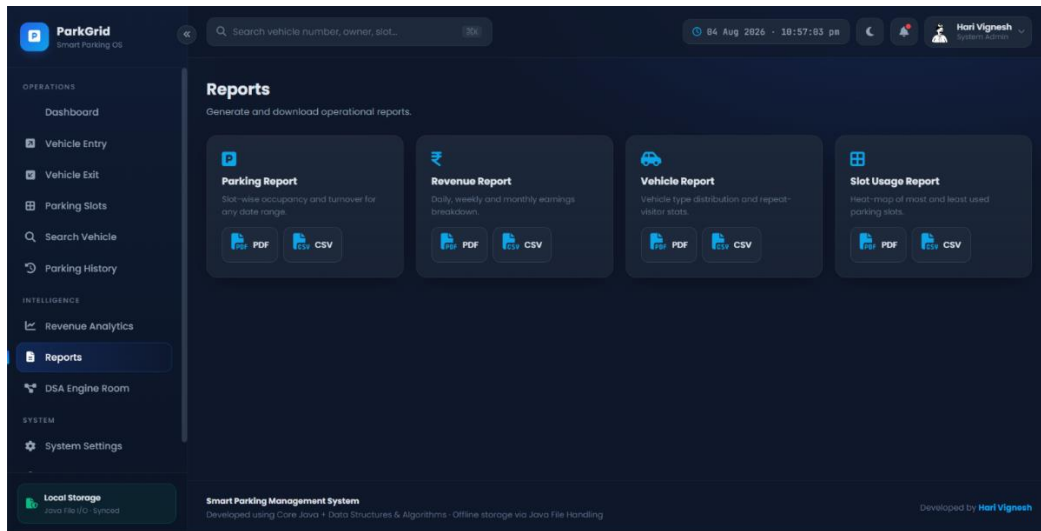
```
<section class="page" id="page-revenue">
  <div class="page-head"><div><h1>Revenue Analytics</h1><p>Where the money is coming from, and when.</p></div></div>

  <div class="kpi-grid kpi-grid-4">
    <div class="kpi-card"><div class="kpi-icon green"><i class="fa-solid fa-indian-rupee-sign"></i></div><div class="kpi-meta"><span class="kpi-label">TODAY</span><span class="kpi-label">₹18,640</span></div></div>
    <div class="kpi-card"><div class="kpi-icon blue"><i class="fa-solid fa-calendar-week"></i></div><div class="kpi-meta"><span class="kpi-label">THIS WEEK</span><span class="kpi-label">₹1,12,480</span></div></div>
    <div class="kpi-card"><div class="kpi-icon sky"><i class="fa-solid fa-calendar-days"></i></div><div class="kpi-meta"><span class="kpi-label">THIS MONTH</span><span class="kpi-label">₹4,86,200</span></div></div>
    <div class="kpi-card"><div class="kpi-icon amber"><i class="fa-solid fa-calendar"></i></div><div class="kpi-meta"><span class="kpi-label">THIS YEAR</span><span class="kpi-label">₹52,10,900</span></div></div>
  </div>

  <div class="grid-2col">
    <div class="card chart-card">
      <div class="card-head"><h3>Monthly Revenue Trend</h3></div>
      <div class="chart-wrap"><canvas id="revTrendChart"></canvas></div>
    </div>
    <div class="card chart-card">
      <div class="card-head"><h3>Revenue by Vehicle Type</h3></div>
      <div class="chart-wrap"><canvas id="revTypeChart"></canvas></div>
    </div>
  </div>

  <div class="card">
    <h3>Key Metrics</h3>
    <div class="metric-strip">
      <div><span>Peak Revenue Hour</span><span><b>6:00 PM</b> - <b>7:00 PM</b></span></div>
      <div><span>Most Used Slot</span><span><b>A-14</b></span></div>
      <div><span>Avg. Parking Time</span><span><b>2h 14m</b></span></div>
      <div><span>Best Day (Month)</span><span><b>Aug 1 - ₹24,120</b></span></div>
    </div>
  </div>
</section>
```

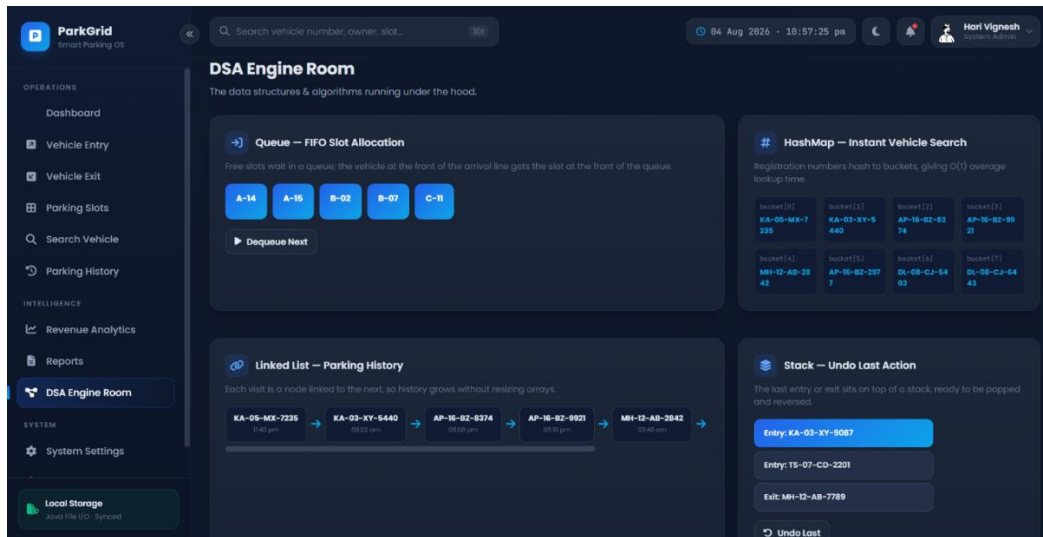
Figure 6.8 Reports:



```
<section class="page" id="page-reports">
  <div class="page-head"><div><h1>Reports</h1><p>Generate and download operational reports.</p></div></div>

  <div class="report-grid">
    <div class="card report-card">
      <i class="fa-solid fa-square-parking"></i>
      <h4>Parking Report</h4>
      <p>Slot-wise occupancy and turnover for any date range.</p>
      <div class="report-actions"><button class="btn btn-secondary sm"><i class="fa-solid fa-file-pdf"></i> PDF</button><button class="btn
    </div>
    <div class="card report-card">
      <i class="fa-solid fa-indian-rupee-sign"></i>
      <h4>Revenue Report</h4>
      <p>Daily, weekly and monthly earnings breakdown.</p>
      <div class="report-actions"><button class="btn btn-secondary sm"><i class="fa-solid fa-file-pdf"></i> PDF</button><button class="btn
    </div>
    <div class="card report-card">
      <i class="fa-solid fa-car-side"></i>
      <h4>Vehicle Report</h4>
      <p>Vehicle type distribution and repeat-visitor stats.</p>
      <div class="report-actions"><button class="btn btn-secondary sm"><i class="fa-solid fa-file-pdf"></i> PDF</button><button class="btn
    </div>
    <div class="card report-card">
      <i class="fa-solid fa-border-all"></i>
      <h4>Slot Usage Report</h4>
      <p>Heat-map of most and least used parking slots.</p>
      <div class="report-actions"><button class="btn btn-secondary sm"><i class="fa-solid fa-file-pdf"></i> PDF</button><button class="btn
    </div>
  </div>
</section>
```

Figure 6.9 DSA Engine Room:



```
<section class="page" id="page-dsa">
  <div class="page-head"><div><h1>DSA Engine Room</h1><p>The data structures & algorithms running under the hood.</p></div></div>

  <div class="dsa-grid">
    <div class="card dsa-card">
      <div class="dsa-head"><i class="fa-solid fa-arrow-right-to-bracket"></i><h3>Queue — FIFO Slot Allocation</h3></div>
      <p>Free slots wait in a queue; the vehicle at the front of the arrival line gets the slot at the front of the queue.</p>
      <div class="queue-viz" id="queueViz"></div>
      <button class="btn btn-secondary sm" id="queueDemoBtn"><i class="fa-solid fa-play"></i> Dequeue Next</button>
    </div>

    <div class="card dsa-card">
      <div class="dsa-head"><i class="fa-solid fa-hashtag"></i><h3>HashMap — Instant Vehicle Search</h3></div>
      <p>Registration numbers hash to buckets, giving  $O(1)$  average lookup time.</p>
      <div class="hashmap-viz" id="hashmapViz"></div>
    </div>

    <div class="card dsa-card">
      <div class="dsa-head"><i class="fa-solid fa-link"></i><h3>Linked List — Parking History</h3></div>
      <p>Each visit is a node linked to the next, so history grows without resizing arrays.</p>
      <div class="linkedList-viz" id="linkedListViz"></div>
    </div>

    <div class="card dsa-card">
      <div class="dsa-head"><i class="fa-solid fa-layer-group"></i><h3>Stack — Undo Last Action</h3></div>
      <p>The last entry or exit sits on top of a stack, ready to be popped and reversed.</p>
      <div class="stack-viz" id="stackViz"></div>
      <button class="btn btn-secondary sm" id="stackDemoBtn"><i class="fa-solid fa-rotate-left"></i> Undo Last</button>
    </div>
  </div>
```

7. Testing and Results

7.1 Test Cases

ID	Test action	Expected outcome	Result
T1	Park a vehicle	Vehicle is assigned an available parking slot	Pass
T2	Search vehicle	Correct vehicle details are displayed	Pass
T3	Remove vehicle	Vehicle is removed and the slot becomes available	Pass
T4	Calculate parking fee	Fee is calculated based on parking duration	Pass
T5	Display parking status	Available and occupied slots are shown correctly	Pass
T6	Add vehicle to waiting queue	Vehicle is added when parking is full	Pass
T7	Save and restart application	Parking records are loaded successfully	Pass
T8	View parking history	All completed parking records are displayed	Pass

Figure 7.1 Successful Vehicle Entry:

Vehicle Entry

Register an incoming vehicle and auto-allocate the nearest free slot.

Entry Details

Vehicle Number

AP30AB1435

Owner Name

Raju

Vehicle Type

EV

Phone Number

9999988888

Nearest slot auto-allocated by Queue DSA

Front of the free-slot queue is dequeued instantly on submit.

Entry Time

10:56 pm

Generate Parking Ticket

Parking Ticket

ParkGrid

Entry Ticket

Slot

B-4

Vehicle No.

AP30AB1435

Owner

Raju

Type

EV

Entry Time

03:59 am



Present this ticket at exit for fee calculation.

Figure 7.2 Successful Search:

Search Vehicle

Instant $O(1)$ lookup powered by a HashMap of registration numbers.

**KA-03-XY-5440**
Priya Nair - Slot A-30 - Entry 08:02 am

Exited

**AP-16-BZ-2977**
Priya Nair - Slot C-25 - Entry 08:57 am

Exited

**TS-07-CD-5778**
Priya Nair - Slot B-1 - Entry 08:44 am

Exited

**TS-07-CD-7639**
Priya Nair - Slot B-23 - Entry 09:29 pm

Exited

**KA-03-XY-7874**
Priya Nair - Slot C-5 - Entry 09:53 pm

Exited

Figure 7.4 Parking History:

VEHICLE NO.	OWNER	TYPE	SLOT	ENTRY TIME	EXIT TIME	DURATION	FEE	STATUS
KA-05-MX-7235	Divya Patel	EV	A-8	11:40 pm	—	—	—	Parked
KA-03-XY-5440	Priya Nair	Bike	A-30	08:02 am	10:19 pm	3h 39m	₹34	Exited
AP-16-BZ-8374	Sneha Iyer	Bike	B-24	06:08 pm	10:19 pm	3h 29m	₹34	Exited
AP-16-BZ-9921	Ananya Das	Truck	C-28	06:10 pm	05:58 pm	3h 28m	₹115	Exited
MH-12-AB-2842	Arjun Reddy	Bike	A-23	03:40 am	06:55 pm	0h 33m	₹10	Exited
AP-16-BZ-2977	Priya Nair	Car	C-25	08:57 am	10:49 pm	3h 22m	₹65	Exited
DL-08-CJ-5403	Kavya Shetty	EV	D-22	10:56 am	—	—	—	Parked
DL-08-CJ-8443	Rohan Gupta	Bike	C-21	03:31 pm	—	—	—	Parked
KA-03-XY-4437	Karthik Manan	Bike	B-24	09:28 pm	10:08 pm	0h 28m	₹10	Exited
TS-07-CD-5778	Priya Nair	EV	B-1	08:44 am	07:12 pm	0h 49m	₹20	Exited

7.2 Observations

The Smart Parking Management System successfully performs all major parking operations through a simple console interface. Vehicle information can be stored, searched, and removed efficiently using appropriate data structures. The Queue manages parking slots effectively, the HashMap provides fast vehicle lookup, and the LinkedList maintains parking history. Local file storage ensures that parking records are preserved between application sessions, making the system reliable and suitable for academic demonstrations.

8. Limitations and Future Scope

8.1 Current Limitations

- The application is designed for a single user and operates on a single system.
- It provides a console-based interface without a graphical user interface.
- Parking records are stored locally and cannot be accessed from multiple devices.
- The system does not support online parking reservation or real-time slot monitoring.

8.2 Future Enhancements

- Develop a graphical user interface using Java Swing or JavaFX.
- Add online parking reservation and cancellation features.
- Integrate RFID or QR code technology for faster vehicle entry and exit.
- Support multi-level parking management with real-time slot availability.
- Connect the application to a database for improved data management and scalability.
- Generate detailed reports and analytics for parking usage and revenue.

9.Conclusion

The Smart Parking Management System demonstrates how Data Structures and Algorithms can be applied to solve a real-world parking management problem. The application provides an efficient way to allocate parking slots, search parked vehicles, calculate parking fees, and maintain parking records through a simple Java console interface.

The use of a Queue for parking slot allocation, a HashMap for fast vehicle search, a LinkedList for maintaining parking history, and a Stack for undo operations improves the overall efficiency of the system. Searching and sorting algorithms further enhance data organization and retrieval. Although the current implementation is console-based, the project provides a strong foundation for future enhancements such as a graphical interface, database integration, and online parking management.

10.References

1. Oracle. Java Platform, Standard Edition Documentation. <https://docs.oracle.com/en/java/>
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*. MIT Press.
3. Oracle. Java Collections Framework Documentation. <https://docs.oracle.com/javase/tutorial/collections/>
4. Smart Parking Management System – Project Source Code and Documentation.

GIT HUB LINK: <https://github.com/harivigneshpericharla/Smart-Parking-Managment>